

Model development with nlmixr2

R/Pharma 2026

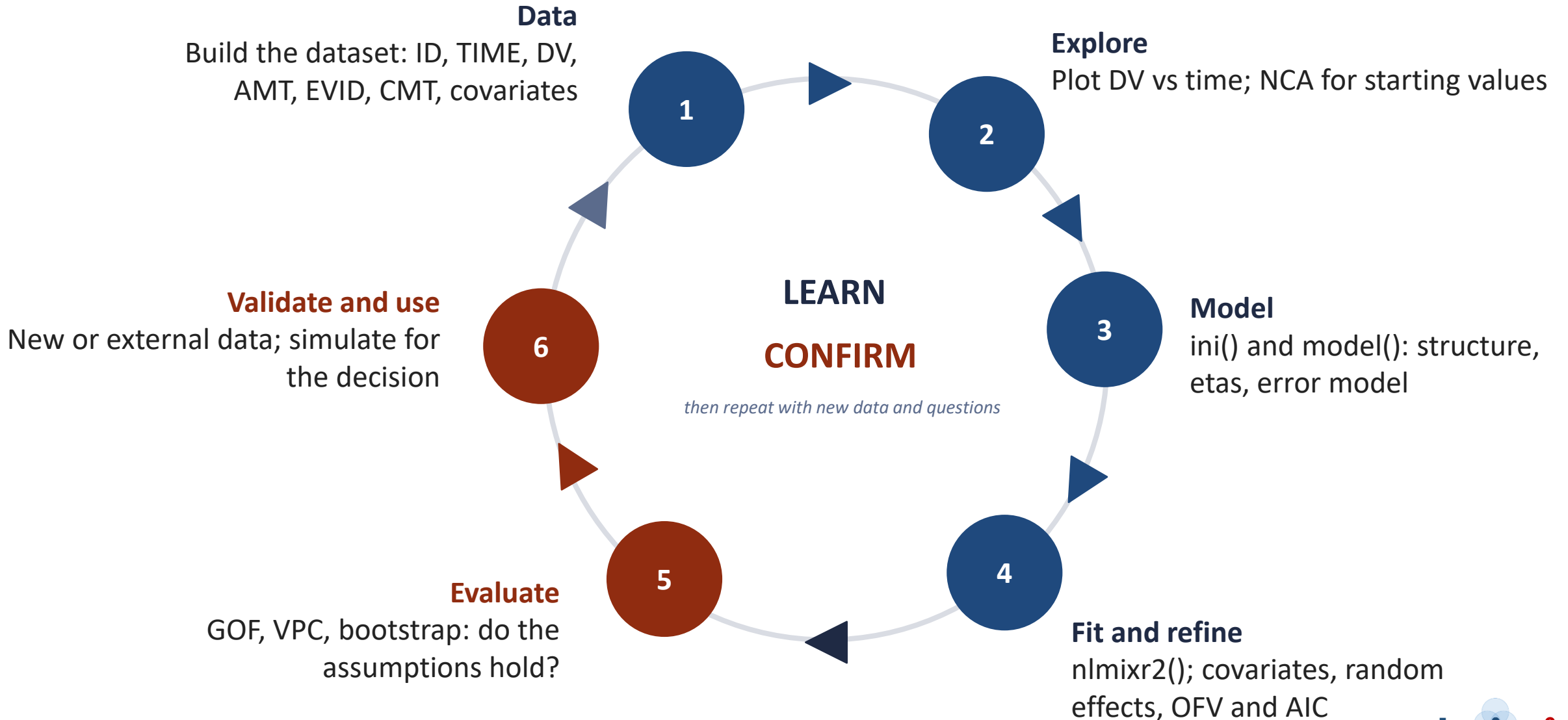
Anne Keunecke, PhD

On behalf of the **nlmixr** development team:

Matt Fidler, Bill Denney, John Harrold, Richard Hooijmaijers, Mutaz Jaber, Anne Keunecke, Theo Papathanasiou, Rik Schoemaker, Max Taubert, Mirjam Trame, Justin Wilkins, Brendan Bender



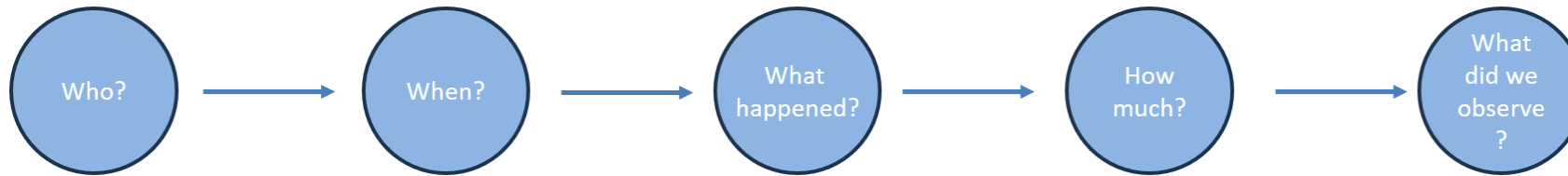
The learn-confirm cycle in nlmixr2 – from data to model in a single environment



Typical structure of a pharmacometric dataset

- Each row represents a single event for a patient (a dose given or an observation recorded).
- Columns generally answer, in order:

ID	TIME	AMT	DV	EVID	WT	AGE	SEX	SPARSE
1	0	100	0	1	66.7	50	1	0
1	0.5	0	0	0	66.7	50	1	0
1	1	0	1.9	0	66.7	50	1	0
1	2	0	3.3	0	66.7	50	1	0
1	3	0	6.6	0	66.7	50	1	0
1	6	0	9.1	0	66.7	50	1	0

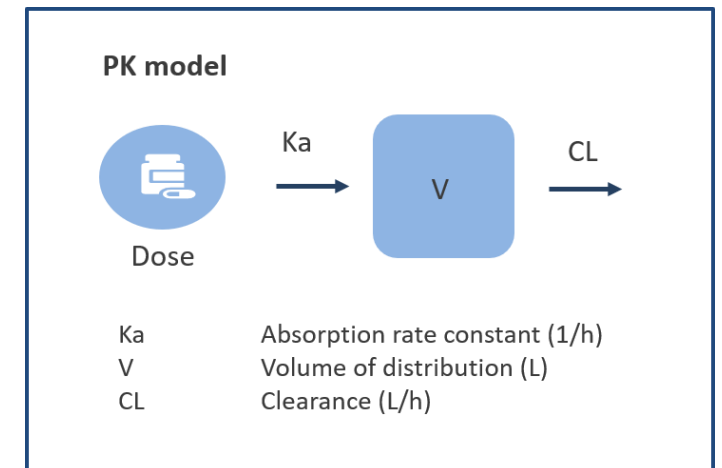


- Minimal required columns
 - ID - unique subject identifier
 - TIME - continuous and relative to the first event within a subject
 - AMT - amount of drug administered
 - DV - dependent variable.
 - EVID - event identifier, 0 = Observation, 1 = Dose, 2 = Other
- Details: <https://nlmixr2.github.io/rxode2/articles/rxode2-event-types.html>

Defining nlmixr2 models

- Models are defined using a function containing an initialisation block (**ini**) and a model definition block (**model**)

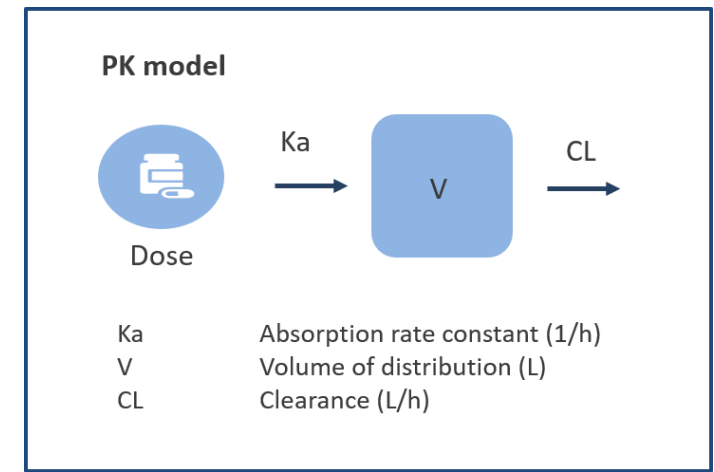
```
One.comp.KA.solved <- function() {  
  ini({  
    # Where initial conditions/variables are specified  
  })  
  model({  
    # Where the model is specified  
  })  
}
```



Defining nlmixr2 models

- The **ini** block defines the parameters
 - Thetas and residual error defined using assignment operators (**<-** or **=**)
 - Etas defined using a model formula (**~**)
- Parameter names, starting values, labels for thetas and errors (using **#**), bounds for some estimation routines (like FOCEI)

```
One.comp.KA.solved <- function() {  
  ini({  
    # Where initial conditions/variables are specified  
    lka <- log(1.15) #log ka (1/h)  
    lcl <- log(0.135) #log CL (L/h)  
    lv <- log(8) #log V (L)  
    prop.err <- 0.15 #proportional error (SD/mean)  
    add.err <- 0.6 #additive error (mg/L)  
    eta.ka ~ 0.5  
    eta.cl ~ 0.1  
    eta.v ~ 0.1  
  })  
  model({  
    ...  
  })  
}
```

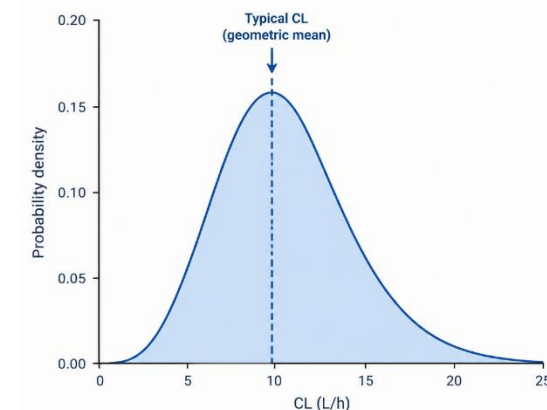
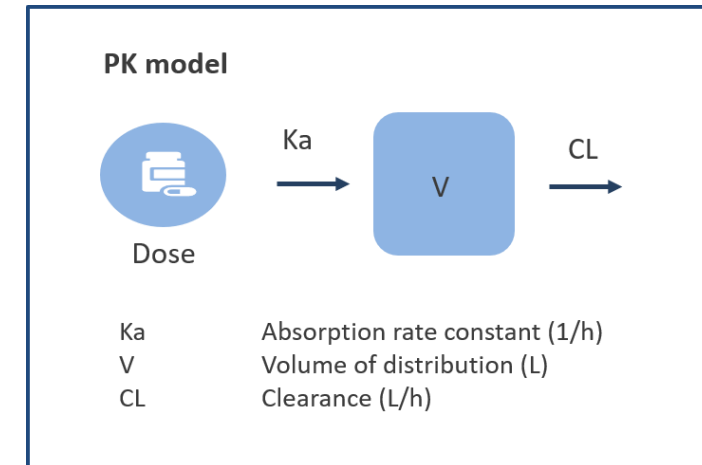


Defining nlmixr2 models

- The **model** block defines
 - the relationship between thetas and etas
 - the model structure using either closed-form solutions or ODEs
 - the residual error structure, and where it is applied

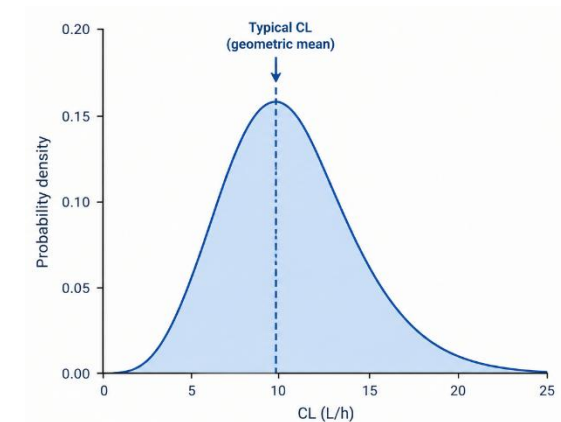
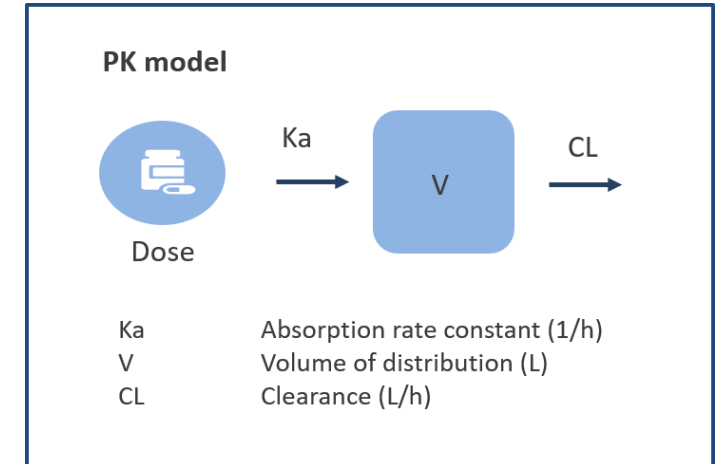
Closed-form solutions are provided by the **linCmt()** function, which infers the model based on the parameters provided

```
One.comp.KA.solved <- function() {  
  ini({  
    ...  
  })  
  model({  
    # Where the model is specified  
    cl <- exp(lcl + eta.cl)  
    v <- exp(lv + eta.v)  
    ka <- exp(lka + eta.ka)  
    ## solved system example  
    ## where residual error is assumed to follow proportional and additive error  
    linCmt() ~ prop(prop.err) + add(add.err)  
  })  
}
```



Running nlmixr2 models: the full model

```
One.comp.KA.solved <- function() {  
  ini({  
    # Where initial conditions/variables are specified  
    lka <- log(1.15) # log ka (1/h)  
    lcl <- log(0.135) # log CL (L/h)  
    lv <- log(8) # log V (L)  
    prop.err <- 0.15 # proportional error (SD/mean)  
    add.err <- 0.6 # additive error (mg/L)  
    eta.ka ~ 0.5  
    eta.cl ~ 0.1  
    eta.v ~ 0.1  
  })  
  model({  
    # Where the model is specified  
    cl <- exp(lcl + eta.cl)  
    v <- exp(lv + eta.v)  
    ka <- exp(lka + eta.ka)  
    ## solved system example  
    ## where residual error is assumed to follow proportional and additive error  
    linCmt() ~ prop(prop.err) + add(add.err)  
  })  
}
```



Assess the data before you start analysis

- Classic data sets used by Nick Holford in his courses:
 - O'Reilly RA, Aggeler PM. Studies on coumarin anticoagulant drugs initiation of warfarin therapy without a loading dose. Circulation 1968;38:169-177
 - O'Reilly RA, Aggeler PM, Leong LS. Studies of the coumarin anticoagulant drugs: The pharmacodynamics of warfarin in man. Journal of Clinical Investigation 1963;42(10):1542-1551
- Some simple plots to get a feel of the data
- Additional **xgxr** functionality to improve and summarize the profiles

Solved systems and ODEs...

- Using solved-system code:

```
linCmt() ~ prop(prop.err) + add(add.err)
```

- For a solved system, model structure is automatically derived from the parameter names in the **ini** block

- Using ODEs:

```
# rxode2-style differential equation definition  
d/dt(depot)      = -ka * depot  
d/dt(central) = ka * depot - (cl / v) * central  
## Concentration is calculated  
cp = central / v  
# And is assumed to follow proportional and additive error  
cp ~ prop(prop.err) + add(add.err)
```

- ODEs are much more flexible but also more time-consuming

mu-referencing and allometric scaling

- For a standard allometric equation we would use:
 - $CL_i = CL_{Pop} \cdot (WT_i/70)^{3/4} \cdot e^\eta$
- Take the log on both sides
 - $\log[CL_i] = \log[CL_{Pop}] + \log\left[(WT_i/70)^{3/4}\right] + \log[e^\eta]$
 - $\log[CL_i] = \log[CL_{Pop}] + 3/4 \cdot \log[WT_i/70] + \eta$
- And back-transforming:
 - $CL_i = e^{\log[CL_{Pop}] + 3/4 \cdot \log[WT_i/70] + \eta}$

Model evaluation in nlmixr2

R/Pharma 2026

Anne Keunecke, PhD

On behalf of the **nlmixr** development team:

Matt Fidler, Bill Denney, John Harrold, Richard Hooijmaijers, Mutaz Jaber, Anne Keunecke, Theo Papathanasiou, Rik Schoemaker, Max Taubert, Mirjam Trame, Justin Wilkins, Brendan Bender



Why do we need to evaluate the models?

“All models are wrong, but some are useful.”

George E. P. Box

Evaluation asks: how wrong is it, and is it useful enough for the decision we need to make?

Questions to challenge/evaluate every model.

Can the model be trusted?

- Converged and stable to initial estimates
- Precise, plausible estimates (RSE, CI)
- Not over-parameterized; shrinkage acceptable
- Bootstrap agrees with final estimates

Is it sufficient for the question?

- Decision and required accuracy defined
- VPC/NPDE match the data where it matters
- Uncertainty small enough for the decision
- Predictions stay within the data range

Are assumptions violated?

- Structure and covariate forms appropriate
- Etas normal, residual error model fits
- Independence, BLQ and outliers handled
- No subject drives the results

Do we see bias?

- DV vs PRED/IPRED symmetric
- CWRES/NPDE flat vs time and covariates
- Etas show no covariate trends
- Subgroups and VPC percentiles unbiased

Diagnostics packages around nlmixr2

Core: the nlmixr2 family

- **nlmixr2plot**: `plot(fit)`, `vpcPlot()`
- **nlmixr2est**: CWRES, NPDE, shrinkage
- **nlmixr2extra**: bootstrap, covariate search

Graphical diagnostics

- **xpose.nlmixr2**: xpose GOF and individual plots (by Justin Wilkins and colleagues)
- **ggPMX**: template plots, automated reports (by colleagues sponsored by Novartis)
- **vpc**: visual prediction plots from nlmixr2 (by Ron Keizer and Bill Denney)
- **tidyvpc**: visual prediction plots from nlmixr2 (by James Craig and team at Cetara)

PKPD models in nlmixr2

R/Pharma 2026

Anne Keunecke, PhD

On behalf of the **nlmixr** development team:

Matt Fidler, Bill Denney, John Harrold, Richard Hooijmaijers, Mutaz Jaber, Anne Keunecke, Theo Papathanasiou, Rik Schoemaker, Max Taubert, Mirjam Trame, Justin Wilkins, Brendan Bender



PKPD modelling in nlmixr2: coupling PK and PD

nlmixr2 supports all three in the same framework, so you can start simple and move to a joint model

Approach	What is fixed / estimated	Typical use
Sequential, IPP (individual PK parameters)	EBEs from the PK fit are merged into the data as covariates; PD is estimated without PK random effects	Fast and robust; good first step (approach on the previous slide)
Sequential, PPP&D (population PK parameters and data)	nlmixr² development team Population PK parameters fixed; PK and PD data both in the model, so individual PK is re-estimated	Accounts for PK uncertainty without refitting all PK parameters
Simultaneous	PK and PD parameters estimated together, with joint random effects and multiple endpoints	Most informative and consistent; slower, needs good starting values

PKPD modelling in nlmixr2: PD model building blocks

Any of these PD structures can be combined with any PK model and any coupling

PD model	Structure	Use when
Direct effect	$E = E_0 + E_{\max} \cdot C^\gamma / (EC_{50}^\gamma + C^\gamma)$ (linear, $E_{\max}$, sigmoid $E_{\max}$)	Effect follows concentration without delay
Effect compartment (link)	$dC_e/dt = k_{e0} \cdot (C - C_e)$; effect driven by C_e	Delayed effect, hysteresis loop
Indirect response (types I–IV)	$dR/dt = k_{in} \cdot (1 - I_{\max} \cdot C / (IC_{50} + C)) - k_{out} \cdot R$, or stimulation/inhibition of k_{out}	Effect on production or loss of a response variable (biomarker turnover)
Extensions	Transit compartments, precursor pool, tolerance/feedback, disease progression, baseline models	Delayed or adaptive responses, long-term studies

Why this is easy in nlmixr2: everything is just model code

- PK via `linCmt()` or ODEs; PD as extra `d/dt()` equations or algebraic expressions in one `model({})` block
- Multiple endpoints with separate error models, e.g. `cp ~ prop(prop.sd)` and `eff ~ add(add.sd)`, linked via DVID or CMT
- Same PD code for sequential and simultaneous fits: only the source of PK parameters changes (data columns vs. estimated)

nlmixr2 is very flexible

You write the model you need; the same code base handles estimation and simulation

Any model structure

Write PK and PD as ODEs or algebraic expressions in one `model({})` block, or use `linCmt()` for standard PK. Direct, effect compartment, indirect response, transit, tolerance, disease progression: all are just equations.

Any coupling

Sequential with EBEs as data, population parameters fixed, or fully simultaneous. Switching is a change in where the PK parameters come from, not a rewrite of the PD model.

Any endpoints and error models

Several observations per subject (PK and PD) with their own error models, linked by DVID or CMT. Additive, proportional and combined errors, plus custom likelihoods for other data types.

One workflow, several tools

Choose an estimation method (for example FOCEi or SAEM), add covariates and IIV/IOV as needed, then simulate and run VPCs from the same model via `rxode2` and R.

PKPD analysis with nlmixr2: sequential estimation

- First approach: use EBEs from a previous PK model to define PK profiles and estimate PKPD relationship
- Extract EBEs from nlmixr2 object and merge to PKPD data file

```
load(file = "run007F.Rdata")
# Use the one compartment transit model to start the PD analysis
EBEs <- as.data.table(run007F)
EBEs <- EBEs[!duplicated(ID), .(ID = as.numeric(as.character(ID)),
  IKTR = ktr, ICL = cl, IV = v)]
PKPDdata <- fread("warfarin_dat.csv")

# Change variable names to upper case (not strictly necessary)
setnames(PKPDdata, names(dataF), toupper(names(dataF)))

# Generate MDV data items
PKPDdata[, MDV := ifelse(is.na(DV), 1, 0)]
PKPDdata[, MDV := ifelse(AMT > 0, 1, MDV)]

# Merge data with PK EBEs
PDdata <- merge(PKPDdata, EBEs, by = "ID", all.x = TRUE)

# remove PK measurements from file so as to not confuse the vpcPlot later on
PDdata[, DEL := ifelse(DVID == 1 & AMT == 0, 1, 0)]
Pddata <- PDdata[DEL == 0]
```

PKPD analysis with nlmixr2: define turnover model using PK EBEs

```
KA1tr1IPP_PDtoemax1 <- function() {  
  ini({  
    tc50 <- log(1)    # Log ec50 (mg/L)  
    tkout <- log(0.05) # Log tkout (/h)  
    te0 <- log(100)   # Log e0  
    eta.c50 ~ .5  
    eta.kout ~ .1  
    eta.e0 ~ .1  
    eps.pdadd <- 100  
  })  
  model({  
    c50 = exp(tc50 + eta.c50)  
    kout = exp(tkout + eta.kout)  
    e0 = exp(te0 + eta.e0)  
    # PK parameters from input data set  
    ktr = IKTR  
    cl = ICL  
    v = IV  
    cp = central/v  
    PD = 1 - cp/(c50 + cp)  
    effect(0) = e0  
    kin = e0 * kout  
    d/dt(depot) = -ktr * depot  
    d/dt(central) = ktr * trans - cl/v * central  
    d/dt(trans) = ktr * depot - ktr * trans  
    d/dt(effect) = kin * PD - kout * effect  
    effect ~ add(eps.pdadd)  
  })  
}
```

Immediate effect simultaneous PKPD analysis with nlmixr2: `ini` block

```
# Immediate effect
KA1tr1_PDimmemax1 <- function() {
  ini({
    ## PK
    tktr <- log(1)   # log ktr (/h)
    tcl  <- log(0.1) # log CL (L/h)
    tv   <- log(8)   # log Vc (L)
    eta.ktr ~ 1
    eta.cl ~ 0.1
    eta.v ~ 0.1
    eps.pkprop <- 0.1 # proportional error (SD/mean)
    eps.pkadd <- 0.4  # additive error (mg/L)

    ## PD
    tc50 <- log(1)   #log ec50 (mg/L)
    te0  <- log(100) #log e0
    eta.c50 ~ .5
    eta.e0 ~ .1
    eps.pdadd <- 100
  })
  model({
  })
}
```

Immediate effect simultaneous PKPD analysis with nlmixr2: `model` block

```
#Immediate effect
model({
  ktr <- exp(tktr + eta.ktr)
  cl  <- exp(tc1 + eta.cl)
  v   <- exp(tv + eta.v)

  c50 = exp(tc50 + eta.c50)
  e0  = exp(te0 + eta.e0)

  cp          = central/v
  d/dt(depot) = -ktr * depot
  d/dt(central) = ktr * trans - cl * cp
  d/dt(trans)  = ktr * depot - ktr * trans
  effect      = e0 * (1 - cp/(c50 + cp))

  cp ~ prop(eps.pkprop) + add(eps.pkadd) | central
  effect ~ add(eps.pdadd) | effect
})
}
```

Effect compartment simultaneous PKPD analysis with nlmixr2: `ini` block

```
#Effect compartment model
KA1tr1_PDceemax <- function() {
  ini({
    ## PK
    tktr <- log(1)    # log ktr (/h)
    tcl  <- log(0.1)  # log CL (L/h)
    tv   <- log(8)    # log Vc (L)
    eta.ktr ~ 1
    eta.cl ~ 0.1
    eta.v ~ 0.1
    eps.pkprop <- 0.1 #proportional error (SD/mean)
    eps.pkadd <- 0.4  #additive error (mg/L)

    ## PD
    tc50 <- log(1)    #log ec50 (mg/L)
    tkout <- log(0.05) #log tkout (/h)
    te0  <- log(100)  #log e0
    eta.c50 ~ .5
    eta.kout ~ .1
    eta.e0 ~ .1
    eps.pdadd <- 100
  })
  model({
  })
}
```

Effect compartment simultaneous PKPD analysis with nlmixr2: `model` block

```
#Effect compartment model
model({
  ktr <- exp(tktr + eta.ktr)
  cl  <- exp(tc1 + eta.cl)
  v   <- exp(tv + eta.v)

  c50 = exp(tc50 + eta.c50)
  kout = exp(tkout + eta.kout)
  e0 = exp(te0 + eta.e0)
  emax = 1

  cp          = central/v
  d/dt(depot) = -ktr * depot
  d/dt(central) = ktr * trans - cl * cp
  d/dt(trans) = ktr * depot - ktr * trans
  d/dt(ce) = kout * (cp - ce)

  effect = e0 * (1 - emax * ce/(c50 + ce))

  cp ~ prop(eps.pkprop) + add(eps.pkadd) | central
  effect ~ add(eps.pdadd) | effect
})
}
```


Turnover model simultaneous PKPD analysis with nlmixr2: ini block

```
# Turnover simultaneous PKPD model
KA1tr1IPP_PDtoemax <- function() {
  ini({
    ## PK
    tktr <- log(1) # log ktr (/h)
    tcl  <- log(0.1) # log CL (L/h)
    tv   <- log(8)  # log Vc (L)
    eta.ktr ~ 1
    eta.cl ~ 0.1
    eta.v ~ 0.1
    eps.pkprop <- 0.1 # proportional error (SD/mean)
    eps.pkadd <- 0.4 # additive error (mg/L)

    ## PD
    tc50 <- log(1) #log ec50 (mg/L)
    tkout <- log(0.05) #log tkout (/h)
    te0 <- log(100) #log e0
    eta.c50 ~ .5
    eta.kout ~ .1
    eta.e0 ~ .1
    eps.pdadd <- 100
  })
  model({
  })
}
```

Turnover model simultaneous PKPD analysis with nlmixr2: `model` block

```
# Turnover simultaneous PKPD model
model({
  ktr <- exp(tktr + eta.ktr)
  cl  <- exp(tc1 + eta.cl)
  v   <- exp(tv + eta.v)
  c50 = exp(tc50 + eta.c50)
  kout = exp(tkout + eta.kout)
  e0   = exp(te0 + eta.e0)
  emax = 1

  cp          = central/v
  d/dt(depot) = -ktr * depot
  d/dt(central) = ktr * trans - cl * cp
  d/dt(trans)  = ktr * depot - ktr * trans
  effect(0)    = e0
  kin          = e0 * kout
  PD           = 1 - emax * cp/(c50 + cp)
  d/dt(effect) = kin * PD - kout * effect

  cp ~ prop(eps.pkprop) + add(eps.pkadd) | central
  effect ~ add(eps.pdadd) | effect
})
}
```